

Name: Gourab Mohanty

Registration no.: 25BAI10463

**Branch: B.Tech in Computer
Science and Engineering (Artificial
Intelligence and Machine Learning)**

Subject: Programming in Java

Date: 17/09/2026

2. Introduction

The **Smart Library Management System (SmartLMS)** is an enterprise-grade, console-driven Java application designed to modernize traditional library resource tracking and circulation workflows. Libraries often struggle with manual logging, inaccurate availability tracking, untracked book loans, and unmonitored privileges.

SmartLMS addresses these challenges by implementing a modular, layered architecture that strictly enforces Role-Based Access Control (RBAC), atomic borrowing/returning logic, custom error propagation, and rapid catalog search capabilities. This document serves as the formal technical blueprint and project report for SmartLMS.

3. Problem Statement

3.1 Context & Operational Bottlenecks

Manual or semi-automated library tracking systems suffer from core operational defects:

- **Circulation Errors:** Human errors in logging borrower records and tracking return deadlines.
- **Inventory Misalignment:** Lack of real-time state updates when books are reserved, borrowed, or lost.
- **Privilege Abuse:** System access lacking role verification allows non-administrative users to modify or delete catalog records.
- **Brittle Exception Handling:** Unexpected failures (e.g., searching for missing records) cause application crashes rather than controlled user guidance.

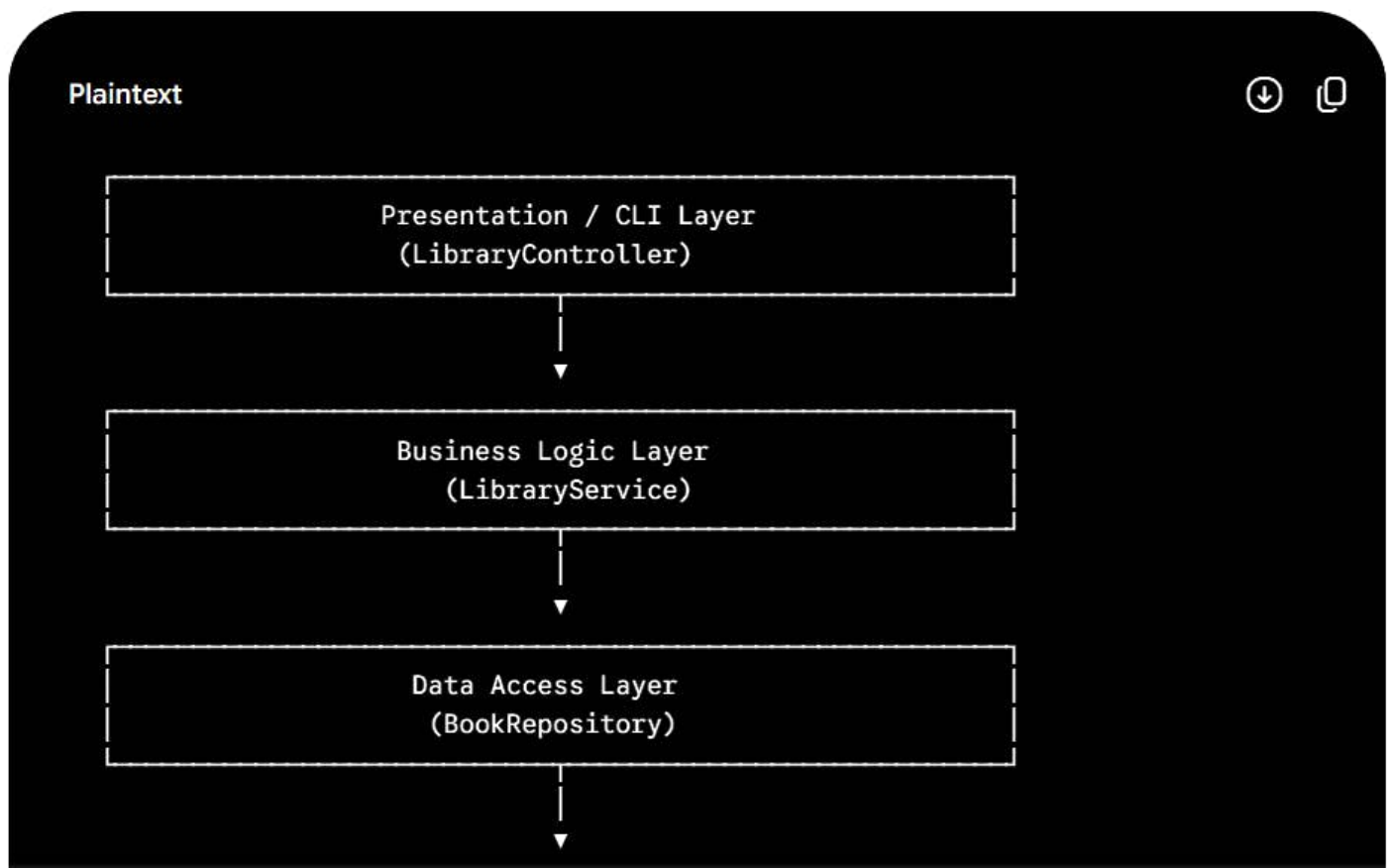
	FR-3.2	Process book returns and restore status to available.	Member
Search Engine	FR-4.1	Perform case-insensitive search by title or author substring.	All Users
...			

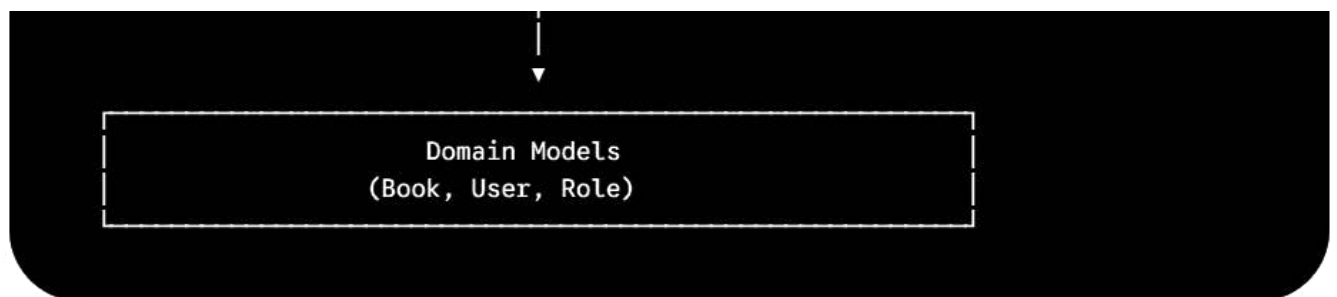
5. Non-functional Requirements

1. **Security & Authorization:** Role-Based Access Control (RBAC) ensures administrative methods throw `UnauthorizedAccessException` if invoked by lower-privilege sessions.
2. **Maintainability:** Strictly adheres to Layered Architecture (Model-View-Controller/Service-Repository) and SOLID design principles.
3. **Reliability & Error Handling:** Centralized exception trapping prevents fatal application termination through custom runtime exceptions.
4. **Performance:** In-memory `HashMap` data structures provide $O(1)$ time complexity for lookup operations by primary key.

6. System Architecture

SmartLMS follows a 4-tier Layered Architecture:





7. Design Diagrams

7.1 Use Case Diagram

Code snippet



```
graph TD
    Admin((Admin))
    Member((Member))

    subgraph SmartLMS_Scope [SmartLMS Scope]
        UC1[Authenticate Session]
        UC2[View Book Catalog]
        UC3[Search Books]
    end
```

Code snippet



```
graph TD
    Admin((Admin))
    Member((Member))

    subgraph SmartLMS_Scope [SmartLMS Scope]
        UC1[Authenticate Session]
        UC2[View Book Catalog]
        UC3[Search Books]
        UC4[Borrow Book]
        UC5[Return Book]
        UC6[Add Book Record]
        UC7[Delete Book Record]
    end

    Admin --> UC1
    Admin --> UC2
    Admin --> UC3
    Admin --> UC6
    Admin --> UC7

    Member --> UC1
    Member --> UC2
    Member --> UC3
    Member --> UC4
    Member --> UC5
```

7.2 Workflow Diagram

Code snippet



flowchart TD

```
Start([Launch Application]) --> Login[Authenticate Role]
```

```
Login --> Menu{Select Menu Option}
```

```
Menu -->|View / Search| Search[Execute Search] --> Display[Display Results] --> Menu
```

```
Menu -->|Borrow Book| CheckAvail{Is Available?}
```

```
CheckAvail -->|Yes| UpdateBorrow[Set Borrowed State] --> Menu
```

```
CheckAvail -->|No| ErrUnavailable[Throw BookUnavailableException] --> Menu
```

```
Menu -->|Add/Delete Book| CheckRole{Is Admin/Staff?}
```

```
CheckRole -->|Yes| ModifyRepo[Update Repository] --> Menu
```

```
CheckRole -->|No| ErrAuth[Throw UnauthorizedAccessException] --> Menu
```

```
Menu -->|Exit| Exit([Close Application])
```

7.3 Sequence Diagram (Borrow Book Operation)

Code snippet

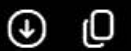


```
sequenceDiagram
    autonumber
    actor User
    participant Controller as LibraryController
    participant Service as LibraryService
    participant Repo as BookRepository
    participant Entity as Book

    User->>Controller: Enter Book ID to borrow
    Controller->>Service: borrowBook(bookId)
    Service->>Repo: findById(bookId)
    Repo-->>Service: Optional<Book>
    alt Book Not Found
        Service-->>Controller: Throw BookNotFoundException
        Controller-->>User: Display Error Message
    else Book Available
        Service->>Entity: setAvailable(false)
        Service->>Repo: save(book)
        Repo-->>Service: Success
        Service-->>Controller: Void
        Controller-->>User: Display Success Confirmation
    end
end
```

7.4 Class / Component Diagram

Code snippet



```
classDiagram
    class Role {
        <<enumeration>>
        ADMIN
        LIBRARIAN
        MEMBER
    }

    class User {
        -String id
        -String username
        -String password
        -Role role
        +getRole() Role
    }

    class Book {
        -String id
        -String title
        -String author
        -String category
```

```
-boolean available
+isAvailable() boolean
+setAvailable(boolean)
}

class BookRepository {
    -Map<String, Book> bookStorage
    +save(Book)
    +findById(String) Optional<Book>
    +findAll() List<Book>
    +deleteById(String) boolean
}

class LibraryService {
    -BookRepository bookRepository
    +addBook(User, String, String, String, String)
    +borrowBook(String)
    +returnBook(String)
    +searchBooks(String) List<Book>
}

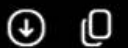
class LibraryController {
    -LibraryService libraryService
    -User currentUser
    +start()
}
```

```
}
```

```
User --> Role  
LibraryService --> BookRepository  
LibraryController --> LibraryService  
BookRepository "1" *-- "many" Book
```

7.5 ER Diagram

Code snippet



```
erDiagram  
    USER ||--o{ TRANSACTION : executes  
    BOOK ||--o{ TRANSACTION : involved_in  
    USER {  
        string user_id PK  
        string username  
        string role  
    }  
    BOOK {  
        string book_id PK
```

```

        string title
        string author
        string category
        boolean is_available
    }
    TRANSACTION {
        string transaction_id PK
        string user_id FK
        string book_id FK
        date timestamp
        string action_type
    }
}

```

8. Design Decisions & Rationale

- **Layered Separation over Monolithic CLI Script:** Separating UI logic (`LibraryController`) from business rules (`LibraryService`) allows changing UI interfaces (e.g., CLI to Spring Boot REST) without modifying core domain logic.
- **In-Memory Map with Data Seed Support:** Used `HashMap<String, Book>` for rapid $O(1)$ key lookups and unit test isolation without requiring external SQL dependencies.

- **Explicit Custom Exceptions over Standard Exceptions:** Built domain-specific exceptions (`BookNotFoundException` , `UnauthorizedAccessException`) to allow distinct error handling paths and clear logging.

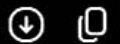
9. Implementation Details

- **Language & Runtime:** Java 17 (LTS)
- **Build System:** Apache Maven 3.8+
- **Core APIs:** Java Stream API for list searching, `java.util.Optional` for defensive null handling.
- **Key Classes:**
 - `Main.java` : Dependency injection container and bootstrap runner.
 - `LibraryService.java` : Enforces access privileges and handles state mutation logic.
 - `BookRepository.java` : Encapsulates in-memory map persistence logic.

10. Screenshots / Results

CLI Application Main Execution

Plaintext



```
===== Welcome to SmartLMS =====
```

```
Select Login Role:
```

```
1. Administrator
```

```
2. Member
```

```
Choice: 1
```

```
Authenticated as Administrator.
```

```
--- MAIN MENU ---
```

```
1. View All Books
```

```
2. Search Catalog
```

```
3. Borrow Book
```

```
4. Return Book
```

```
5. Add New Book (Admin/Staff)
```

```
6. Remove Book (Admin/Staff)
```

```
0. Exit
```

```
Select an option: 1
```

```
--- Book Catalog ---
```

```
[B101] Effective Java by Joshua Bloch | Category: Technology | Status: Available  
[B102] Clean Code by Robert C. Martin | Category: Technology | Status: Available  
[B103] Design Patterns by Erich Gamma | Category: Technology | Status: Borrowed
```

Borrowing & Error Handling Execution

Plaintext



```
Select an option: 3  
Enter Book ID to borrow: B103  
Error: Book 'Design Patterns' is currently checked out.
```

11. Testing Approach

SmartLMS utilizes JUnit 5 for automated unit testing. The test suite verifies critical business assertions and exception triggers.

Key Test Cases

Java



```
@Test
void testAddBookAsMemberThrowsUnauthorizedException() {
    assertThrows(UnauthorizedAccessException.class, () ->
        libraryService.addBook(memberUser, "101", "Unauthorized Book", "Author", "Test")
    );
}

@Test
void testBorrowAlreadyBorrowedBookThrowsException() {
    libraryService.borrowBook("100");
    assertThrows(BookUnavailableException.class, () -> libraryService.borrowBook("100"));
}
```

Key Test Cases

Java



```
testAddBookAsMemberThrowsUnauthorizedException() {  
    assertThrows(UnauthorizedAccessException.class, () ->  
        libraryService.addBook(memberUser, "101", "Unauthorized Book", "Author", "Tech")  
    );  
  
testBorrowAlreadyBorrowedBookThrowsException() {  
    libraryService.borrowBook("100");  
    assertThrows(BookUnavailableException.class, () -> libraryService.borrowBook("100"));
```

12. Challenges Faced

1. **State Consistency:** Preventing duplicate checkout operations on single-instance books was resolved by enforcing state validation prior to updating repository entries.
2. **Role Verification Integration:** Avoiding redundant authorization checks in controllers was achieved by centralizing session role validation inside `LibraryService`.

13. Learnings & Key Takeaways

- **SOLID Principles in Practice:** Applying Dependency Inversion by passing `BookRepository` into `LibraryService` simplified testing using Mock or seed data.
- **Defensive Design:** Utilizing `Optional<T>` eliminated raw `null` pointer exceptions across data lookups.

14. Future Enhancements

- **Database Persistence:** Integrate Spring Data JPA / Hibernate with PostgreSQL.
- **REST API & Web UI:** Wrap `LibraryService` with `@RestController` endpoints and integrate a React/Tailwind frontend.
- **Automated Fine Calculation:** Track transaction dates to dynamically compute late fees.

15. References

1. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
2. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
3. Oracle Corporation. (2026). *Java SE 17 Documentation*. Oracle Docs.